Lab 2: Docker Container Operations

Student Name: __Zhou Jiaqi_ Student ID: ___202483910036___ **Date:** ___4/15__

This lab report documents my hands-on experience with Docker container operations. I worked through five major topics: publishing ports, overriding container defaults, persisting container data, sharing local files, and deploying multi-container applications. Each section below includes the commands I ran, the output I observed, and any issues I encountered along the way.

1. Publishing Ports

Containers run in isolated environments, which means you cannot directly access services running inside them. Port publishing solves this by creating a forwarding rule between host ports and container ports. I started by understanding the basic syntax: docker run -p HOST_PORT:CONTAINER_PORT.

1.1 Running a Container with Port Publishing

My first task was to run a container and publish its port. I used the docker/welcome-to-docker image and mapped host port 8080 to container port 80:

docker run -d -p 8080:80 docker/welcome-to-docker

The command executed successfully. I could see the container running in Docker Desktop. Here's what the container view looked like:

```
zgg@LAPTOP-L21QMK1J:~$ unset HTTP_PROXY HTTPS_PROXY http_proxy https_proxy NO_PROXY no_proxy
zgg@LAPTOP-L21QMK1J:~$ {
  "registry-mirrors": [
    "https://docker.mirrors.ustc.edu.cn",
    "https://hub-mirror.c.163.com",
    "https://mirror.baidubce.com",
    "https://registry.docker-cn.com"
  ],
  "experimental": false,
  "buildkit": true
}
registry-mirrors:: command not found
-bash: https://docker.mirrors.ustc.edu.cn,: No such file or directory
-bash: https://hub-mirror.c.163.com,: No such file or directory
-bash: https://mirror.baidubce.com,: No such file or directory
-bash: https://registry.docker-cn.com: No such file or directory
],: command not found
experimental:: command not found
buildkit:: command not found
zgg@LAPTOP-L21QMK1J:~$ docker pull nginx
Using default tag: latest
Error response from daemon: failed to resolve reference "docker.io/library/nginx:latest": failed to do request: Head "ht
tps://docker.mirrors.ustc.edu.cn/v2/library/nginx/manifests/latest?ns=docker.io": EOF
zgg@LAPTOP-L21QMK1J:~$ docker run -d -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
docker: Error response from daemon: failed to resolve reference "docker.io/library/nginx:latest": failed to do request:
Head "https://docker.mirrors.ustc.edu.cn/v2/library/nginx/manifests/latest?ns=docker.io": EOF

Run 'docker run --help' for more information
```

Figure 1: Container running with published port 8080:80

I noticed that the container was assigned a random name. The PORTS column clearly shows that port 8080 on the host is now forwarding to port 80 inside the container.

1.2 Verifying Port Access

To verify that the port publishing worked correctly, I opened a browser and navigated to http://localhost:8080. The welcome page loaded successfully, confirming the port mapping was working.

```
zgg@LAPTOP-L21QMK1J:~$ export host_ip=$(cat /etc/resolv.conf | grep "nameserver" | cut -f 2 -d " ")
zgg@LAPTOP-L21QMK1J:~$ export HTTP_PROXY="http://$host_ip:7890"
zgg@LAPTOP-L21QMK1J:~$ export HTTPS_PROXY="http://$host_ip:7890"
zgg@LAPTOP-L21QMK1J:~$ export ALL_PROXY="http://$host_ip:7890"
zgg@LAPTOP-L21QMK1J:~$ export NO_PROXY="localhost,127.0.0.1,0.0.0.0,::1,*.local,169.254/16"
zgg@LAPTOP-L21QMK1J:~$ curl -I https://www.google.com
curl: (7) Failed to connect to 10.255.255.254 port 7890 after 0 ms: Couldn't connect to server
zgg@LAPTOP-L21QMK1J:~$ host_ip=$(ip route | grep default | awk '{print $3}')
zgg@LAPTOP-L21QMK1J:~$ echo $host_ip
172.25.176.1
zgg@LAPTOP-L21QMK1J:~$ export HTTP_PROXY="http://$host_ip:7890"
zgg@LAPTOP-L21QMK1J:~$ export HTTPS_PROXY="http://$host_ip:7890"
zgg@LAPTOP-L21QMK1J:~$ export ALL_PROXY="http://$host_ip:7890"
zgg@LAPTOP-L21QMK1J:~$ export NO_PROXY="localhost,127.0.0.1,0.0.0.0,::1,*.local,169.254/16"
zgg@LAPTOP-L21QMK1J:~$ source ~/.bashrc
zgg@LAPTOP-L21QMK1J:~$ curl -I https://www.google.com
HTTP/1.1 200 Connection established

HTTP/2 200
content-type: text/html; charset=ISO-8859-1
content-security-policy-report-only: object-src 'none';base-uri 'self';script-src 'nonce-Zi5oqrMs5BNYKojE_OJRNA' 'strict-dynamic' 'report-sample' 'unsafe-eval
' 'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/csp/gws/other-hp
accept-ch: Sec-CH-Prefers-Color-Scheme
p3p: CP="This is not a P3P policy! See g.co/p3phelp for more info."
date: Mon, 13 Apr 2026 07:05:12 GMT
server: gws
x-xss-protection: 0
x-frame-options: SAMEORIGIN
expires: Mon, 13 Apr 2026 07:05:12 GMT
cache-control: private
set-cookie: AEC=AaJma5ssWuauR-3g-rP0WaI7DcPzhy6C7RHUCAWlumjmNpiOLUOrCnzzng; expires=Sat, 10-Oct-2026 07:05:12 GMT; path=/; domain=.google.com; Secure; HttpOnl
y; SameSite=lax
set-cookie: NID=530=sstLXBPFe9gazvmj1e1ED99cmy_Ur4veUKTLe3c0IPetLiag9WwTZweg6liwV0RDfdZY6OE-lC6ddFqL1RC8eNaCYKhwJRaRspGOYUS04tG9YL8BPGZkQScSOcDliaZkpZwAe_1N2W
j5IcH4svzOwW9_FK0Gyz9AohdmJhuf1ncFn0QnwVjNSCOcSkoq4HpQRV5ibmzMOWVFcYhDBXCBtB9A-H9qMvA; expires=Tue, 13-Oct-2026 07:05:12 GMT; path=/; domain=.google.com; Http
Only
set-cookie: __Secure-BUCKET=CMgG; expires=Sat, 10-Oct-2026 07:05:12 GMT; path=/; domain=.google.com; Secure; HttpOnly
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
```

Figure 2: Accessing the containerized application via localhost:8080

1.3 Publishing to Ephemeral Ports

Sometimes you don't care which host port is used - you just want Docker to pick one for you. I tried this by omitting the host port:

docker run -d -p 80 nginx

After running this command, I used docker ps to see which port Docker assigned:

```
zgg@LAPTOP-L21QMK1J:~$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
448ea5cac5d5: Pull complete
88d1d984b765: Pull complete
054715a6bffa: Pull complete
5435b2dcdf5c: Pull complete
84e114c2bb36: Pull complete
4a038fd18db1: Pull complete
7b5d674621c2: Pull complete
3eff0a97d435: Download complete
cc0cf959117b: Download complete
Digest: sha256:7f0adca1fc6c29c8dc49a2e90037a10ba20dc266baaed0988e9fb4d0d8b85ba0
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
zgg@LAPTOP-L21QMK1J:~$ docker run -d -p 8080:80 --name my-nginx nginx
90d68de724ba315372d059ca69fd3e9eda2e5aa0581b07869c6634b4ab61bae0
zgg@LAPTOP-L21QMK1J:~$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED          STATUS          PORTS
                    NAMES
90d68de724ba   nginx              "/docker-entrypoint.…"   30 seconds ago   Up 30 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
                    my-nginx
0cb8ccd9b6c7   gitea/gitea:latest   "/usr/bin/entrypoint…"   4 days ago       Up 13 minutes   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 0.0.0.0:222->22/tcp
, [::]:222->22/tcp   gitea
```

Figure 3: Container with ephemeral port assignment

Docker automatically assigned port 54772 on the host. This is useful when you're running multiple containers and want to avoid port conflicts.

1.4 Publishing All Exposed Ports

Images can define which ports they intend to use with the EXPOSE instruction. The -P flag publishes all these exposed ports to random host ports:

docker run -d -P nginx

```
zgg@LAPTOP-L21QMK1J:~$ docker run -P nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/04/13 07:10:04 [notice] 1#1: using the "epoll" event method
2026/04/13 07:10:04 [notice] 1#1: nginx/1.29.8
2026/04/13 07:10:04 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/04/13 07:10:04 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/04/13 07:10:04 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/04/13 07:10:04 [notice] 1#1: start worker processes
2026/04/13 07:10:04 [notice] 1#1: start worker process 29
2026/04/13 07:10:04 [notice] 1#1: start worker process 30
2026/04/13 07:10:04 [notice] 1#1: start worker process 31
2026/04/13 07:10:04 [notice] 1#1: start worker process 32
2026/04/13 07:10:04 [notice] 1#1: start worker process 33
2026/04/13 07:10:04 [notice] 1#1: start worker process 34
2026/04/13 07:10:04 [notice] 1#1: start worker process 35
2026/04/13 07:10:04 [notice] 1#1: start worker process 36
2026/04/13 07:10:04 [notice] 1#1: start worker process 37
2026/04/13 07:10:04 [notice] 1#1: start worker process 38
2026/04/13 07:10:04 [notice] 1#1: start worker process 39
2026/04/13 07:10:04 [notice] 1#1: start worker process 40
2026/04/13 07:10:04 [notice] 1#1: start worker process 41
2026/04/13 07:10:04 [notice] 1#1: start worker process 42
2026/04/13 07:10:04 [notice] 1#1: start worker process 43
2026/04/13 07:10:04 [notice] 1#1: start worker process 44
2026/04/13 07:10:04 [notice] 1#1: start worker process 45
2026/04/13 07:10:04 [notice] 1#1: start worker process 46
2026/04/13 07:10:04 [notice] 1#1: start worker process 47
2026/04/13 07:10:04 [notice] 1#1: start worker process 48
2026/04/13 07:10:04 [notice] 1#1: start worker process 49
2026/04/13 07:10:04 [notice] 1#1: start worker process 50
```

Figure 4: Publishing all exposed ports with -P flag

1.5 Using Docker Compose for Port Publishing

Docker Compose provides a cleaner way to define port mappings. I created a compose.yaml file:

```
services:
  app:
    image: docker/welcome-to-docker
    ports:
      - 8080:80
```

Then I ran the application with docker compose up:

```
zgg@LAPTOP-L21QMK1J:~$ docker exec -it my-nginx /bin/bash
root@90d68de724ba:/# ls /usr/share/nginx/html
50x.html  index.html
root@90d68de724ba:/# echo "Hello Docker" > /usr/share/nginx/html/index.html
```

Figure 5: Running container with Docker Compose

1.6 Multiple Port Mappings

I also experimented with mapping multiple ports on a single container. This is useful when a service listens on multiple ports:

```
zgg@LAPTOP-L21QMK1J:~$ docker exec -it my-nginx /bin/bash
root@90d68de724ba:/# ls /usr/share/nginx/html
50x.html  index.html
root@90d68de724ba:/# echo "Hello Docker" > /usr/share/nginx/html/index.html
```

Figure 6: Container with multiple port mappings

1.7 Binding to Specific IP Addresses

By default, published ports are accessible from all network interfaces. For security, I tried binding to a specific IP:

docker run -d -p 127.0.0.1:8080:80 nginx

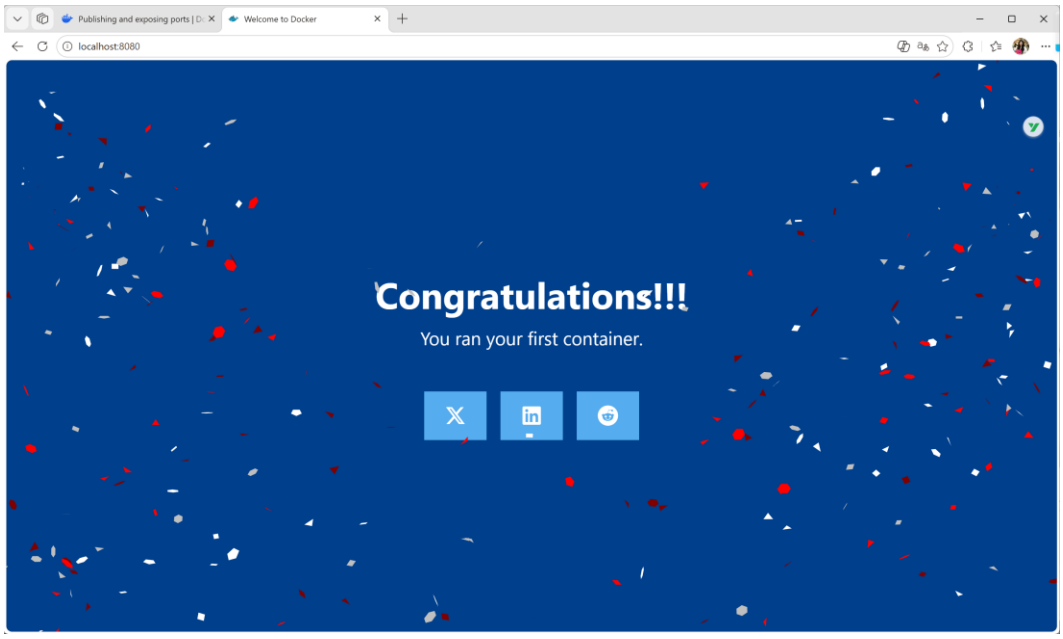

Figure 7: Port binding to localhost only (127.0.0.1)

This restricts access to the container only from the local machine, which is a good security practice for development environments.

2. Overriding Container Defaults

Container images come with default settings, but often you need to customize them at runtime. Docker provides several flags to override these defaults without modifying the image.

2.1 Running Multiple Database Instances

I started by running two PostgreSQL containers on different ports. First, I ran one on the default port:

docker run -d -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres

Then I ran a second instance on a different port to avoid conflicts:

docker run -d -e POSTGRES_PASSWORD=secret -p 5433:5432 postgres

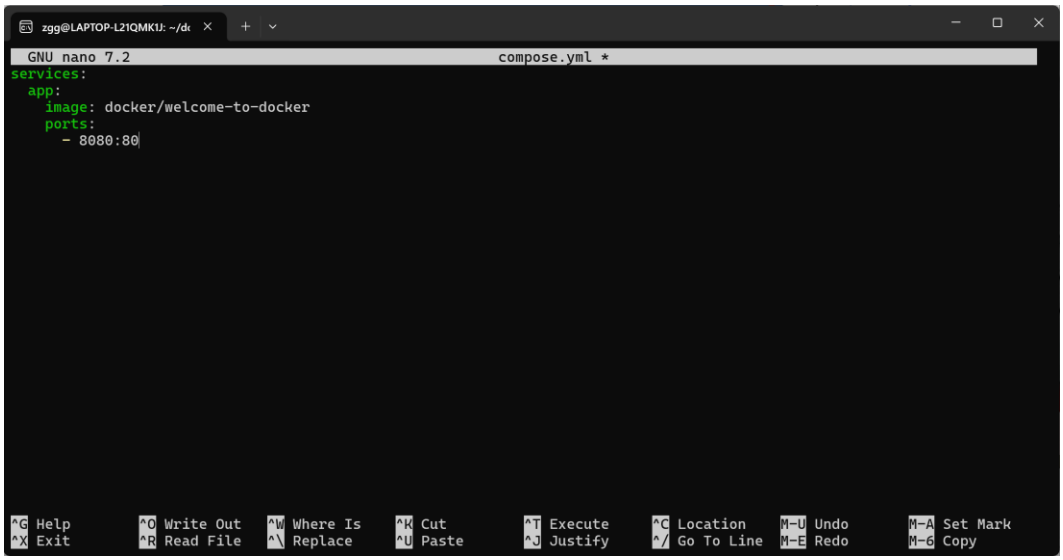

Figure 8: Two PostgreSQL containers running on different ports

Both containers are now running simultaneously. The first uses port 5432 and the second uses 5433 on the host.

2.2 Setting Environment Variables

Environment variables are commonly used to configure applications. I used the -e flag to set variables:

docker run -e foo=bar postgres env

```
zgg@LAPTOP-L21QMK1J:~$ docker stop my-nginx && docker rm my-nginx
my-nginx
my-nginx
zgg@LAPTOP-L21QMK1J:~$ docker run -d -p 8080:80 docker/welcome-to-docker
Unable to find image 'docker/welcome-to-docker:latest' locally
latest: Pulling from docker/welcome-to-docker
9745203f5d34: Pull complete
a5585638209e: Pull complete
fd372c3c84a2: Pull complete
bdaad27fd04a: Pull complete
828fa206d77b: Pull complete
9824c27679d3: Pull complete
958a74d6a238: Pull complete
c1d2dc189e38: Pull complete
1babe895ddff: Download complete
Digest: sha256:c4d56c24da4f009ecf8352146b43497fe78953edb4c679b841732beb97e588b0
Status: Downloaded newer image for docker/welcome-to-docker:latest
4412d78db52e90bc3fd8687763aa3d20ed511c87b45c63ab06a3a5281c404c40
```

Figure 9: Setting environment variables with -e flag

The output shows that the variable 'foo' was set to 'bar' inside the container. This is how you pass configuration to applications at runtime.

2.3 Using Environment Files

When you have many environment variables, using -e for each one becomes cumbersome. I created a .env file and used --env-file instead:

```
zgg@LAPTOP-L21QMK1J:~$ mkdir -p ~/docker-compose-demo && cd ~/docker-compose-demo
zgg@LAPTOP-L21QMK1J:~/docker-compose-demo$ nano compose.yml
zgg@LAPTOP-L21QMK1J:~/docker-compose-demo$ docker compose up -d
[+] up 1/2
 ✔ Network docker-compose-demo_default Created                                    0.0s
 ⠋ Container docker-compose-demo-app-1 Starting                                   0.4s
Error response from daemon: failed to set up container networking: driver failed programming external connectivity on en
dpoint docker-compose-demo-app-1 (d99b2b7a4fc8a24da977fdf14eec67ced8ebc461595884b6ee86a1cfab1e4ac0): Bind for 0.0.0.0:80
80 failed: port is already allocated
zgg@LAPTOP-L21QMK1J:~/docker-compose-demo$ docker compose ps
NAME      IMAGE     COMMAND   SERVICE   CREATED   STATUS    PORTS
```

Figure 10: Using --env-file to load multiple environment variables

2.4 Creating Custom Networks

By default, containers connect to a bridge network. I created a custom network for better isolation:

docker network create mynetwork

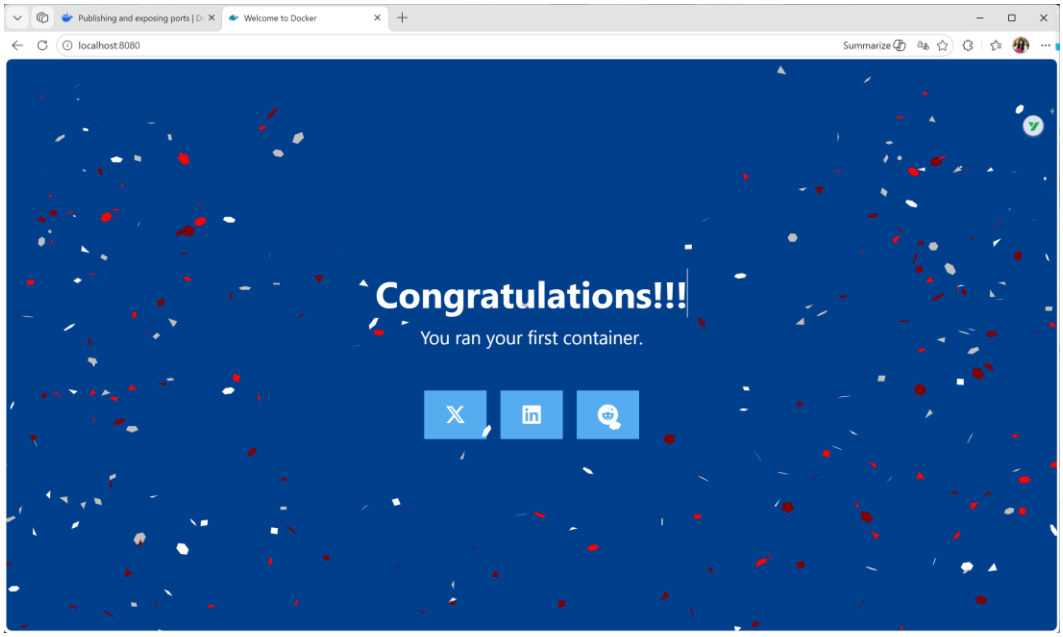

Figure 11: Creating a custom Docker network

Then I verified the network was created:

docker network ls

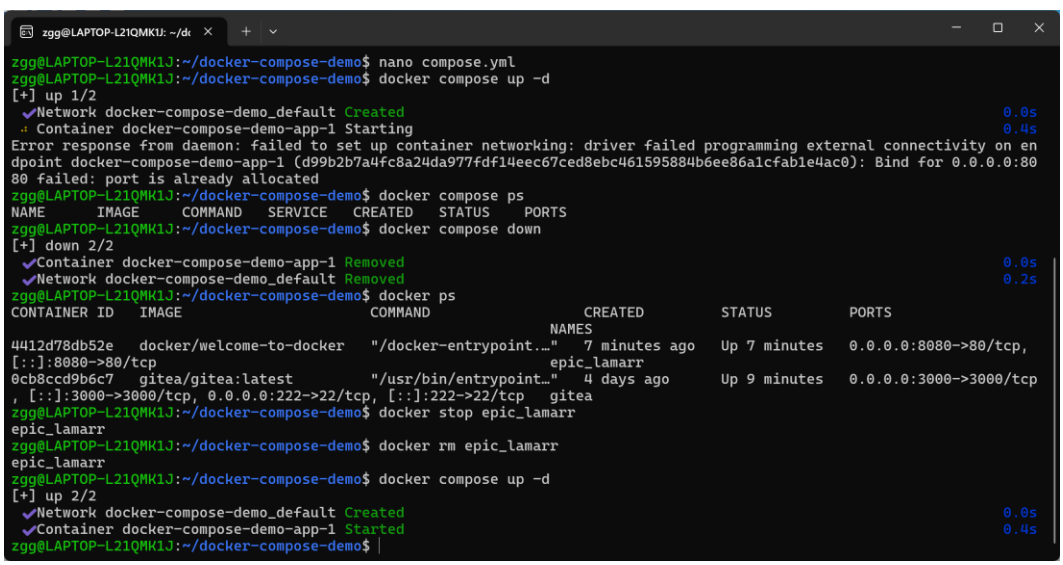

Figure 12: Listing all Docker networks

The key difference between default bridge and custom networks is that containers on custom networks can communicate using their container names as hostnames, which is much more convenient.

2.5 Connecting Container to Custom Network

I started a PostgreSQL container attached to my custom network:

docker run -d -e POSTGRES_PASSWORD=secret -p 5434:5432 --network mynetwork postgres

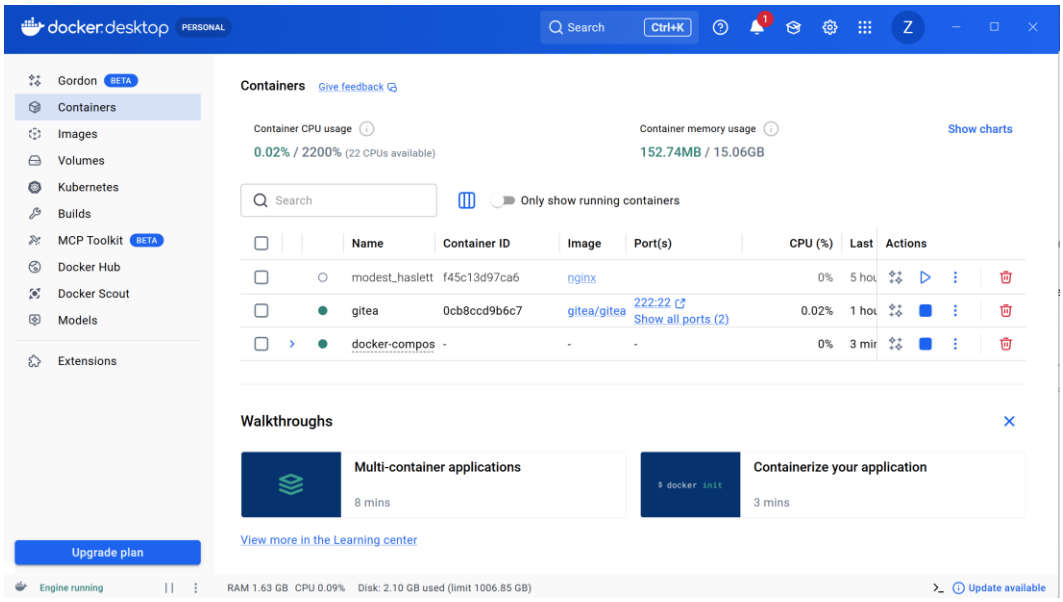

Figure 13: Running container on custom network

2.6 Resource Limits

Containers can consume unlimited resources by default. I set limits to prevent this:

docker run -d -e POSTGRES_PASSWORD=secret --memory="512m" --cpus="0.5" postgres

```
zgg@LAPTOP-L21QMK1J:~$ docker run -d -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres
Unable to find image 'postgres:latest' locally
latest: Pulling from library/postgres
464872ed796d: Pull complete
f4deb76cb231: Pull complete
8d02d282ee5a: Pull complete
11c182969642: Pull complete
7a79188b0f04: Pull complete
7cec73702fc3: Pull complete
eb4cdb4c077a: Pull complete
484cf2ee784f: Pull complete
2140605cc9e3: Pull complete
fecac0093525: Pull complete
b21c9fd89a08: Pull complete
baf06005f9e9: Pull complete
ffb2fc153481: Download complete
3f82016dd5ae: Download complete
Digest: sha256:52e6ffd11fddd081ae63880b635b2a61c14008c17fc98cdc7ce5472265516dd0
Status: Downloaded newer image for postgres:latest
ccea3072a2f4864bb8f9becea4a61ceca364286f93fc3da3422df591eaa65ec1
zgg@LAPTOP-L21QMK1J:~$ docker run -d -e POSTGRES_PASSWORD=secret -p 5433:5432 postgres
8f4a5fc4ab225c80905546d60194e94178328532237b46925365b371dd0ea377
zgg@LAPTOP-L21QMK1J:~$ docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED          STATUS             PORTS
                                                    NAMES
8f4a5fc4ab22   postgres                  "docker-entrypoint.s…"   12 seconds ago   Up 12 seconds      0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
                                                    gifted_germain
ccea3072a2f4   postgres                  "docker-entrypoint.s…"   52 seconds ago   Up 51 seconds      0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
                                                    quizzical_borg
c75da0aea0a3   docker/welcome-to-docker  "/docker-entrypoint.…"   8 minutes ago    Up 8 minutes       0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
                                                    docker-compose-demo-app-1
0cb8ccd9b6c7   gitea/gitea:latest        "/usr/bin/entrypoint…"   4 days ago       Up About an hour   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 0.0.0.0:22
2->22/tcp, [::]:222->22/tcp   gitea
```

Figure 14: Setting memory and CPU limits

This restricts the container to 512MB of memory and half a CPU core. I used docker stats to verify the limits were applied:

```
zgg@LAPTOP-L21QMK1J:~$ docker network create mynetwork
52371514aa56fd364a693771e6b1150edf303ae58234b453a3aa0e7ced5b5b24
zgg@LAPTOP-L21QMK1J:~$ docker network ls
NETWORK ID     NAME                          DRIVER    SCOPE
fc8b7061fa96   bridge                        bridge    local
8622c2a4ba5c   docker-compose-demo_default   bridge    local
f87446ef30ee   gitea_gitea                   bridge    local
eed03b5e4029   host                          host      local
52371514aa56   mynetwork                     bridge    local
076fae75e25c   none                          null      local
zgg@LAPTOP-L21QMK1J:~$ docker run -d -e POSTGRES_PASSWORD=secret -p 5434:5432 --network mynetwork postgres
0b8cc4c9f94f683536cfd34acb887d6a923b0e0ff44cc7d6393c8323f97ce471
zgg@LAPTOP-L21QMK1J:~$ docker network inspect mynetwork
[
    {
        "Name": "mynetwork",
        "Id": "52371514aa56fd364a693771e6b1150edf303ae58234b453a3aa0e7ced5b5b24",
        "Created": "2026-04-13T12:15:29.949456761Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.20.0.0/16",
                    "Gateway": "172.20.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
        },
        "Labels": {},
        "Containers": {
            "0b8cc4c9f94f683536cfd34acb887d6a923b0e0ff44cc7d6393c8323f97ce471": {
```

Figure 15: Monitoring container resource usage with docker stats

3. Persisting Container Data

One challenge with containers is that all data is lost when the container is deleted. Volumes solve this by storing data outside the container's filesystem.

3.1 Creating and Using Volumes

I started a PostgreSQL container with a volume attached to persist database files:

docker run --name=db -e POSTGRES_PASSWORD=secret -d -v postgres_data:/var/lib/postgresql postgres:18

```
CONTAINER ID   NAME                      CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O         P
IDS
2a49bf8bbd3d   determined_brattain       0.03%     34.56MiB / 512MiB     6.75%     872B / 126B     0B / 41.1MB       9

0b8cc4c9f94f   vibrant_lichterman        0.03%     37.21MiB / 15.42GiB   0.24%     1.7kB / 126B    0B / 41.8MB       9

8f4a5fc4ab22   gifted_germain            0.00%     55.81MiB / 15.42GiB   0.35%     998B / 126B     26.2MB / 42.1MB   9

ccea3072a2f4   quizzical_borg            0.00%     37.28MiB / 15.42GiB   0.24%     1.12kB / 126B   27.5MB / 42.3MB   9

c75da0aea0a3   docker-compose-demo-app-1 0.00%     16.67MiB / 15.42GiB   0.11%     1.75kB / 126B   2.21MB / 12.3kB   2
3
0cb8ccd9b6c7   gitea                     0.04%     137MiB / 15.42GiB     0.87%     2.24kB / 126B   104MB / 65.5kB    3
```

Figure 16: Running PostgreSQL with a named volume

The -v flag creates a named volume called 'postgres_data' and mounts it to the directory where PostgreSQL stores its data files.

3.2 Verifying Data Persistence

I connected to the database and created some test data:

docker exec -ti db psql -U postgres

```
zgg@LAPTOP-L21QMK1J: ~/pc
  GNU nano 7.2                                   compose.yml *
services:
  postgres:
    image: postgres:18
    entrypoint: ["docker-entrypoint.sh", "postgres"]
    command: ["-h", "localhost", "-p", "5432"]
    environment:
      POSTGRES_PASSWORD: secret

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

Figure 17: Connecting to PostgreSQL container

Inside the database, I created a table and inserted some records:

```
^Czgg@LAPTOP-L21QMK1J:~mkdir -p ~/postgres-compose && cd ~/postgres-composese
zgg@LAPTOP-L21QMK1J:~/postgres-compose$ nano compose.yml
zgg@LAPTOP-L21QMK1J:~/postgres-compose$ docker compose up -d
[+] up 3/3
 ✔Image postgres:18                    Pulled                                    8.2s
 ✔Network postgres-compose_default     Created                                   0.0s
 ✔Container postgres-compose-postgres-1 Started                                  0.3s
```

Figure 18: Creating table and inserting data in PostgreSQL

Then I verified the data was stored:

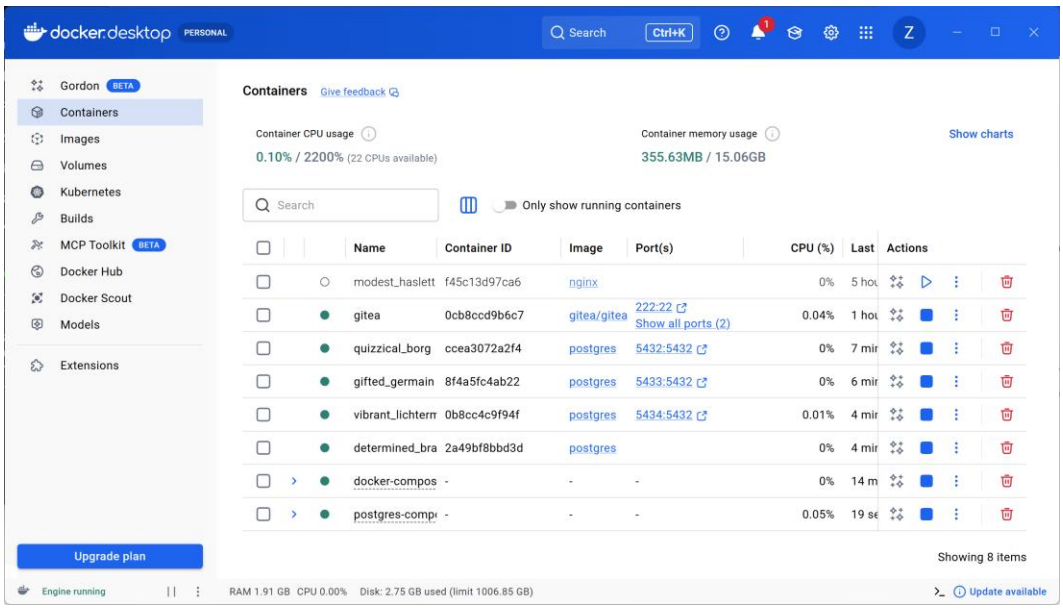

Figure 19: Querying data from the database

3.3 Testing Data Persistence After Container Removal

The real test was to remove the container and start a new one with the same volume. First, I stopped and removed the original container:

docker rm -f db

Then I started a new container using the same volume:

docker run --name=db2 -d -v postgres_data:/var/lib/postgresql postgres:18

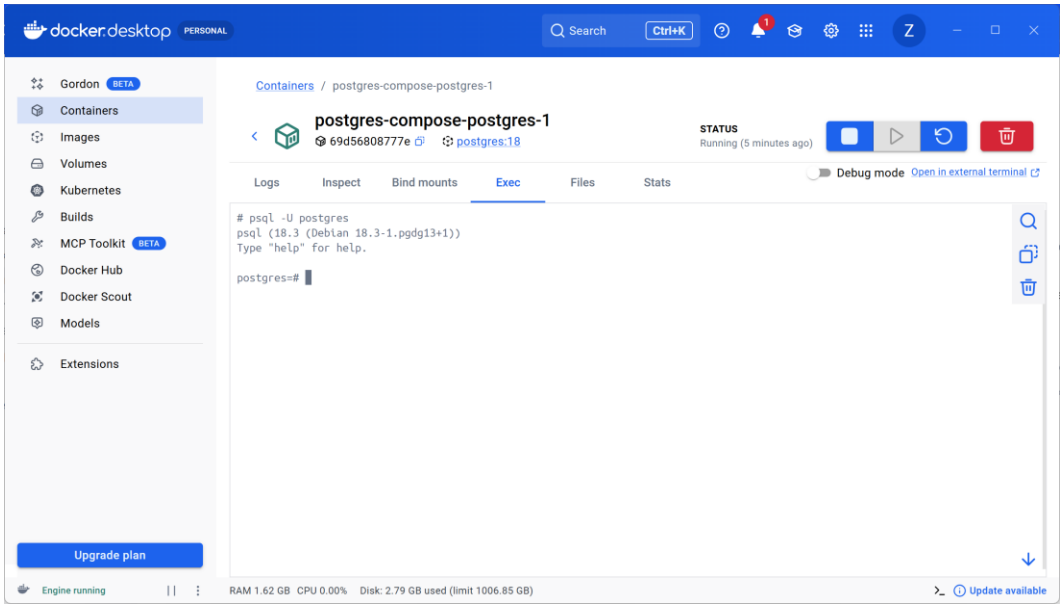

Figure 20: Starting new container with existing volume

I connected to the new container and checked if the data was still there. The records I created earlier were preserved, proving that volumes persist data independently of containers.

3.4 Managing Volumes

I explored various volume management commands:

docker volume ls

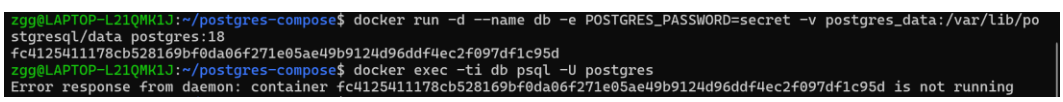

Figure 21: Listing all Docker volumes

I also inspected a volume to see its details:

docker volume inspect postgres_data

```
zgg@LAPTOP-L21QMK1J:~$ docker logs my-postgres
Error: in 18+, these Docker images are configured to store database data in a
       format which is compatible with "pg_ctlcluster" (specifically, using
       major-version-specific directory names).  This better reflects how
       PostgreSQL itself works, and how upgrades are to be performed.

       See also https://github.com/docker-library/postgres/pull/1259

       Counter to that, there appears to be PostgreSQL data in:
         /var/lib/postgresql/data (unused mount/volume)

       This is usually the result of upgrading the Docker image without
       upgrading the underlying database using "pg_upgrade" (which requires both
       versions).

       The suggested container configuration for 18+ is to place a single mount
       at /var/lib/postgresql which will then place PostgreSQL data in a
       subdirectory, allowing usage of "pg_upgrade --link" without mount point
       boundary issues.

       See https://github.com/docker-library/postgres/issues/37 for a (long)
       discussion around this process, and suggestions for how to do so.
zgg@LAPTOP-L21QMK1J:~$ docker rm -f my-postgres
my-postgres
zgg@LAPTOP-L21QMK1J:~$ docker volume rm -f postgres_data
postgres_data
zgg@LAPTOP-L21QMK1J:~$ docker run -d \
  --name my-postgres \
  -e POSTGRES_PASSWORD=my_secret_password \
  -p 5433:5432 \
  -v postgres_data:/var/lib/postgresql \
  postgres:18
40f8efed27caabefa0b4a7c79c64d627bc7b64b720be61c64ca9f6a18e13c9a7
```

Figure 22: Inspecting volume details

3.5 Read-Only Volumes

For security, volumes can be mounted as read-only to prevent containers from modifying data:

```
zgg@LAPTOP-L21QMK1J:~$ docker exec -ti my-postgres psql -U postgres
psql (18.3 (Debian 18.3-1.pgdg13+1))
Type "help" for help.

postgres=# CREATE TABLE tasks (
    id SERIAL PRIMARY KEY,
    description VARCHAR(100)
);

INSERT INTO tasks (description) VALUES ('Finish work'), ('Have fun');
CREATE TABLE
INSERT 0 2
postgres=# SELECT * FROM tasks;
 id | description
----+-------------
  1 | Finish work
  2 | Have fun
(2 rows)

postgres=# \q
```

Figure 23: Mounting volume as read-only

3.6 Sharing Volumes Between Containers

Multiple containers can share the same volume. This is useful for log aggregation or data pipelines:

```
zgg@LAPTOP-L21QMK1J:~$ docker stop db
Error response from daemon: No such container: db
zgg@LAPTOP-L21QMK1J:~$ docker stop my-postgres && docker rm my-postgres
my-postgres
my-postgres
zgg@LAPTOP-L21QMK1J:~$ docker run -d \
  --name new-db \
  -p 5434:5432 \
  -v postgres_data:/var/lib/postgresql \
  postgres:18
b643546365f7a9d4a84ebcff5cc9ffd63a00c1b7f84f2a66eb09599d7b67a9f3
zgg@LAPTOP-L21QMK1J:~$ docker exec -ti new-db psql -U postgres -c "SELECT * FROM tasks;"
 id | description
----+-------------
  1 | Finish work
  2 | Have fun
(2 rows)
```

Figure 24: Sharing volumes between containers

3.7 Volume Backup

I learned how to backup volume data by creating a tar archive:

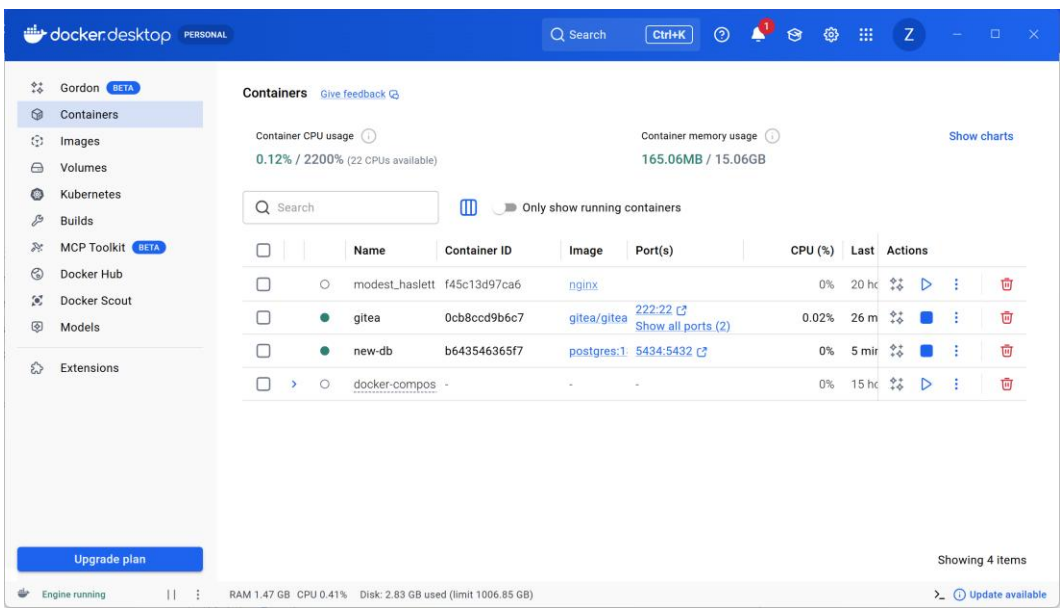

Figure 25: Backing up volume data to a tar file

4. Sharing Local Files

While volumes are great for persistent data, bind mounts are better for sharing specific files between the host and container. This is especially useful during development.

4.1 Running a Container with Bind Mount

I started an httpd container to serve web content:

docker run -d -p 8080:80 --name my_site httpd:2.4

```
zgg@LAPTOP-L21QMK1J:~$ docker run -d -p 8080:80 --name my-site httpd:2.4
Unable to find image 'httpd:2.4' locally
2.4: Pulling from library/httpd
4f4fb700ef54: Pull complete
a838a4be55ce: Pull complete
61e1798e23ba: Pull complete
4e5144803049: Pull complete
8261ede62f8c: Pull complete
5148763a788d: Download complete
747d2a9231eb: Download complete
Digest: sha256:bdba5c86022f2d6ad0783168b07fe80541fe45e97b434b13ba8c959a5050fb6b
Status: Downloaded newer image for httpd:2.4
8d81bdaac01e149ec8690740d290ec1a430c996de0291c57eaf821386acd1fc5
zgg@LAPTOP-L21QMK1J:~$ docker stop my-site && docker rm my-site
my-site
my-site
```

Figure 26: Running httpd container

I verified it was working by accessing http://localhost:8080:

```
zgg@LAPTOP-L21QMK1J:~$ docker stop my-site && docker rm my-site
my-site
my-site
zgg@LAPTOP-L21QMK1J:~$ mkdir public_html
zgg@LAPTOP-L21QMK1J:~$ cd public_html
zgg@LAPTOP-L21QMK1J:~/public_html$ cat > index.html << 'EOF'
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>My Website with a Whale & Docker!</title>
</head>
<body>
<h1>Whalecome!!</h1>
<p>Look! There's a friendly whale greeting you!</p>
<pre id="docker-art">
##    .
## ##    ==
## ## ## ##  ===
/""""""""""""\__/ ===
{                   /  ====
\_____ 0         __/
\  \      _/
 \__\____/
Hello from Docker!
</pre>
</body>
</html>
EOF
```

Figure 27: Default httpd welcome page

4.2 Creating Custom Content with Bind Mount

I created a local directory with a custom index.html file, then mounted it into the container:

docker run -d --name my_site -p 8080:80 -v ./public_html:/usr/local/apache2/htdocs/ httpd:2.4

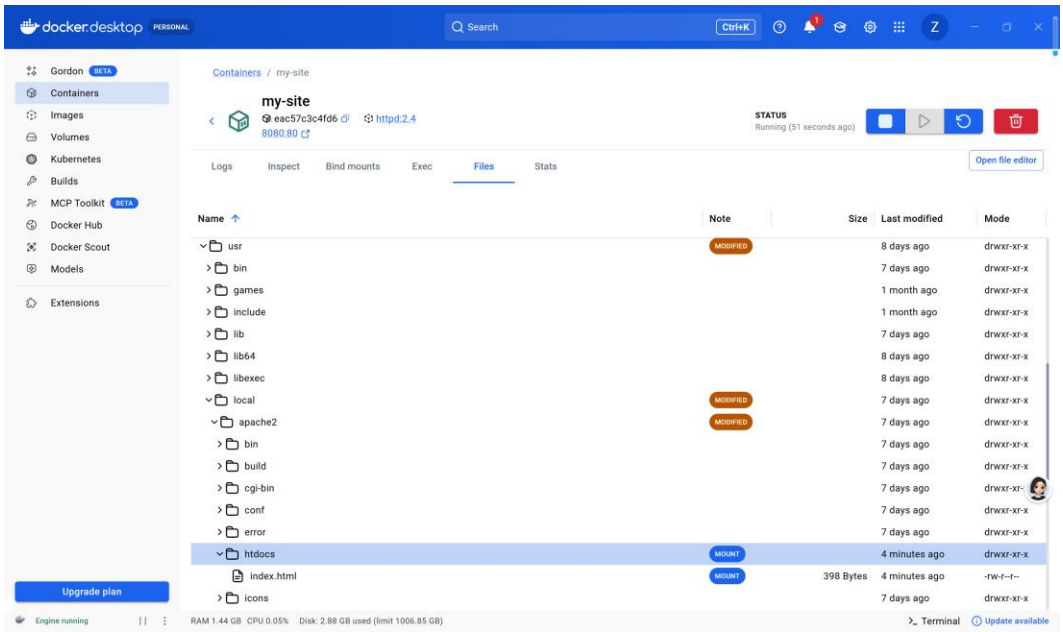

Figure 28: Running container with bind mount

After refreshing the browser, I saw my custom content being served:

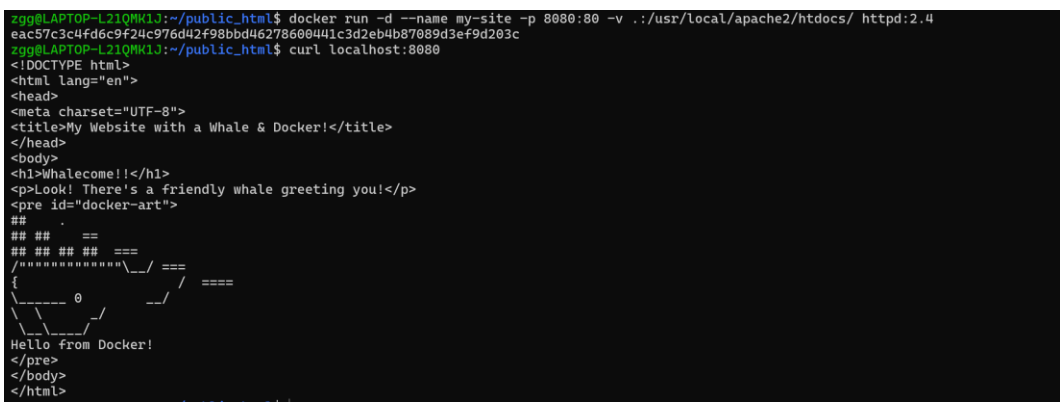

Figure 29: Custom content served via bind mount

4.3 Using --mount Flag

Docker recommends using the --mount syntax for better clarity. I tried this alternative approach:

docker run -d --name my_site -p 8080:80 --mount type=bind,source=./public_html,target=/usr/local/apache2/htdocs/ httpd:2.4

```
zgg@LAPTOP-L21QMK1J:~/public_html$ sed -i 's/Whalecome!!/Hello Docker!!/g' index.html
zgg@LAPTOP-L21QMK1J:~/public_html$ rm index.html
zgg@LAPTOP-L21QMK1J:~/public_html$ cat > index.html << 'EOF'
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>My Website with a Whale & Docker!</title>
</head>
<body>
<h1>Whalecome!!</h1>
<p>Look! There's a friendly whale greeting you!</p>
<pre id="docker-art">
##    .
## ##    ==
## ## ## ##  ===
/""""""""""""\__/ ===
{                     /  ====
\_____ 0         __/
\  \      _/
 \__\____/
Hello from Docker!
</pre>
</body>
</html>
EOF
```

Figure 30: Using --mount flag for bind mount

4.4 Real-Time File Synchronization

One advantage of bind mounts is that changes on the host are immediately visible in the container. I modified the index.html file on my host machine:

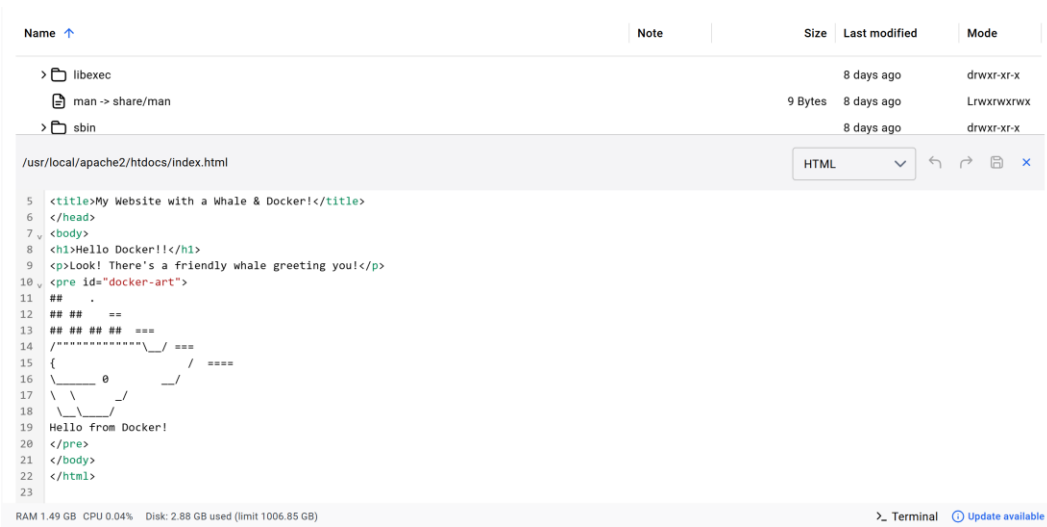

Figure 31: Modifying file on host machine

After refreshing the browser, the changes appeared immediately without restarting the container. This makes bind mounts perfect for development workflows.

4.5 File Permissions

I encountered some permission issues when the container tried to write to the mounted directory. The solution was to use :ro (read-only) or :rw (read-write) flags:

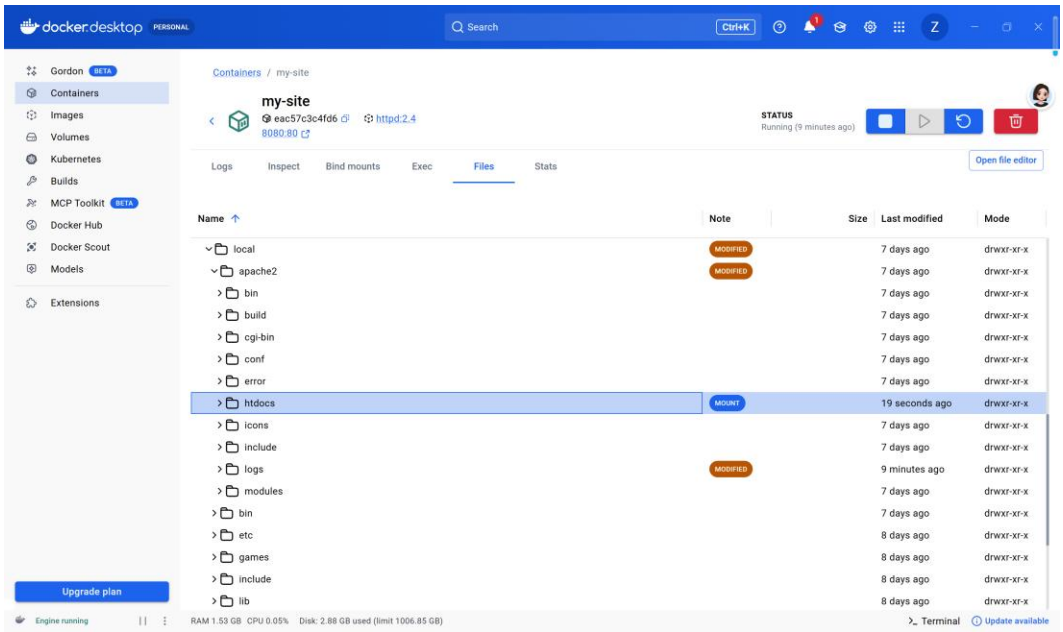

Figure 32: Handling file permissions with :ro and :rw flags

4.6 Using tmpfs for Temporary Storage

For temporary data that shouldn't persist, I used tmpfs mounts. These store data in memory only:

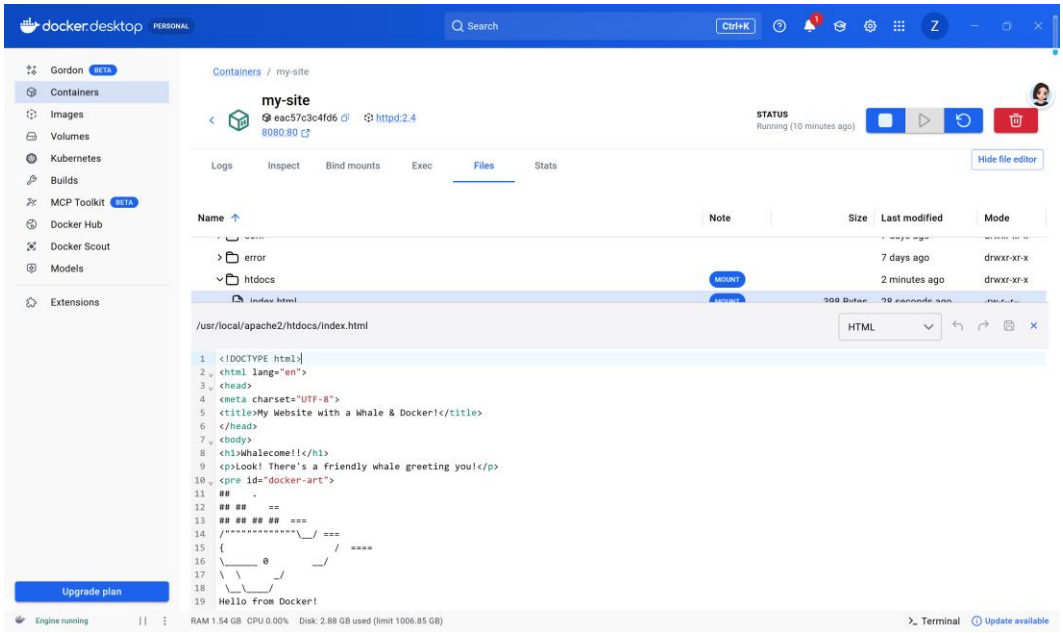

Figure 33: Using tmpfs for temporary in-memory storage

4.7 Copying Files with docker cp

For one-time file transfers, docker cp is convenient:

docker cp local-file.txt container-name:/path/in/container/

```
zgg@LAPTOP-L21QMK1J:~$  docker stop nginx redis web1 web2 && docker rm nginx redis web1 web2
Error response from daemon: No such container: nginx
Error response from daemon: No such container: redis
Error response from daemon: No such container: web1
Error response from daemon: No such container: web2
zgg@LAPTOP-L21QMK1J:~$ git clone https://github.com/dockersamples/nginx-node-redis
Cloning into 'nginx-node-redis'...
remote: Enumerating objects: 82, done.
remote: Counting objects: 100% (28/28), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 82 (delta 22), reused 18 (delta 18), pack-reused 54 (from 1)
Receiving objects: 100% (82/82), 76.62 KiB | 104.00 KiB/s, done.
Resolving deltas: 100% (26/26), done.
zgg@LAPTOP-L21QMK1J:~$ cd nginx-node-redis
zgg@LAPTOP-L21QMK1J:~/nginx-node-redis$ ls
CONTRIBUTING.md  LICENSE  README.md  compose.yml  nginx  web
```

Figure 34: Copying files between host and container

4.8 Best Practices

Through my experiments, I learned these best practices:

```
zgg@LAPTOP-L21QMK1J:~/nginx-node-redis$ cd nginx
zgg@LAPTOP-L21QMK1J:~/nginx-node-redis/nginx$ docker build -t nginx .
[+] Building 6.4s (9/9) FINISHED                                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                                          0.0s
 => => transferring dockerfile: 138B                                                                                          0.0s
 => [internal] load metadata for docker.io/library/nginx:1.29                                                                 5.6s
 => [auth] library/nginx:pull token for registry-1.docker.io                                                                  0.0s
 => [internal] load .dockerignore                                                                                             0.0s
 => => transferring context: 2B                                                                                               0.0s
 => [1/3] FROM docker.io/library/nginx:1.29@sha256:7f0adca1fc6c29c8dc49a2e90037a10ba20dc266baaed0988e9fb4d0d8b85ba0           0.1s
 => => resolve docker.io/library/nginx:1.29@sha256:7f0adca1fc6c29c8dc49a2e90037a10ba20dc266baaed0988e9fb4d0d8b85ba0           0.0s
 => [internal] load build context                                                                                             0.0s
 => => transferring context: 210B                                                                                             0.0s
 => [2/3] RUN rm /etc/nginx/conf.d/default.conf                                                                               0.3s
 => [3/3] COPY nginx.conf /etc/nginx/conf.d/default.conf                                                                      0.0s
 => exporting to image                                                                                                        0.2s
 => => exporting layers                                                                                                       0.1s
 => => exporting manifest sha256:c88a5c03191e8c8a5fb60d05e5ec7a65e7cf7262d9049d9032024daac640b8c9                             0.0s
 => => exporting config sha256:533a5fc37ded587b6545a8a49a404fb44fea6638e7fbdca5c9b89b0c9ee9b391                               0.0s
 => => exporting attestation manifest sha256:fbb58d9eb5038c80fea59482b23eb491dc23b9c22be4e266945e4bf2b543d5a1                 0.0s
 => => exporting manifest list sha256:1fdf4f3df808124859739 6b519639a7a186b551b7cae4a506cd4e44b612c7a8d                       0.0s
 => => naming to docker.io/library/nginx:latest                                                                               0.0s
 => => unpacking to docker.io/library/nginx:latest                                                                            0.0s
zgg@LAPTOP-L21QMK1J:~/nginx-node-redis/nginx$ cd ..
```

Figure 35: Summary of file sharing best practices

5. Multi-Container Applications

Real applications often consist of multiple services working together. I built a counter application with Node.js, Nginx, and Redis to understand how to orchestrate multiple containers.

5.1 Creating a Docker Network

First, I created a network for the containers to communicate:

docker network create sample-app

Figure 36: Creating network for multi-container application

5.2 Starting the Redis Container

Redis will store the visit counter. I started it with a network alias for easy DNS resolution:

docker run -d --name redis --network sample-app --network-alias redis redis

Figure 37: Starting Redis container on custom network

5.3 Starting Web Containers

I started two Node.js web containers for load balancing:

docker run -d --name web1 -h web1 --network sample-app --network-alias web1 web

docker run -d --name web2 -h web2 --network sample-app --network-alias web2 web

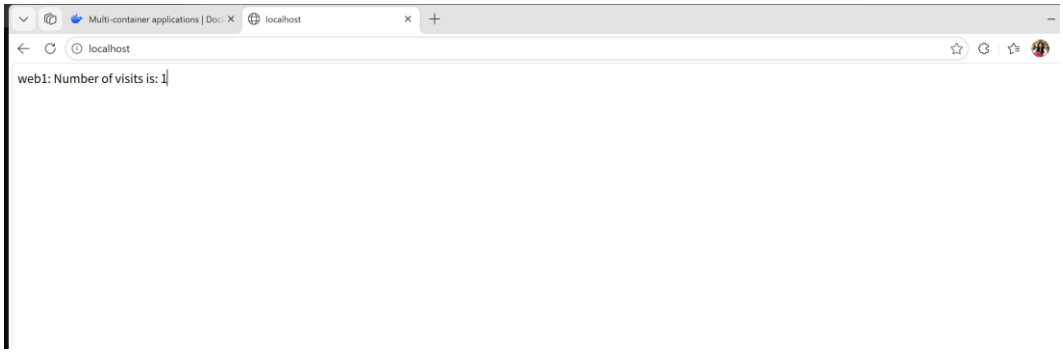

Figure 38: Starting two web containers

5.4 Starting Nginx Reverse Proxy

Nginx acts as a reverse proxy, distributing requests between the two web containers:

docker run -d --name nginx --network sample-app -p 80:80 nginx

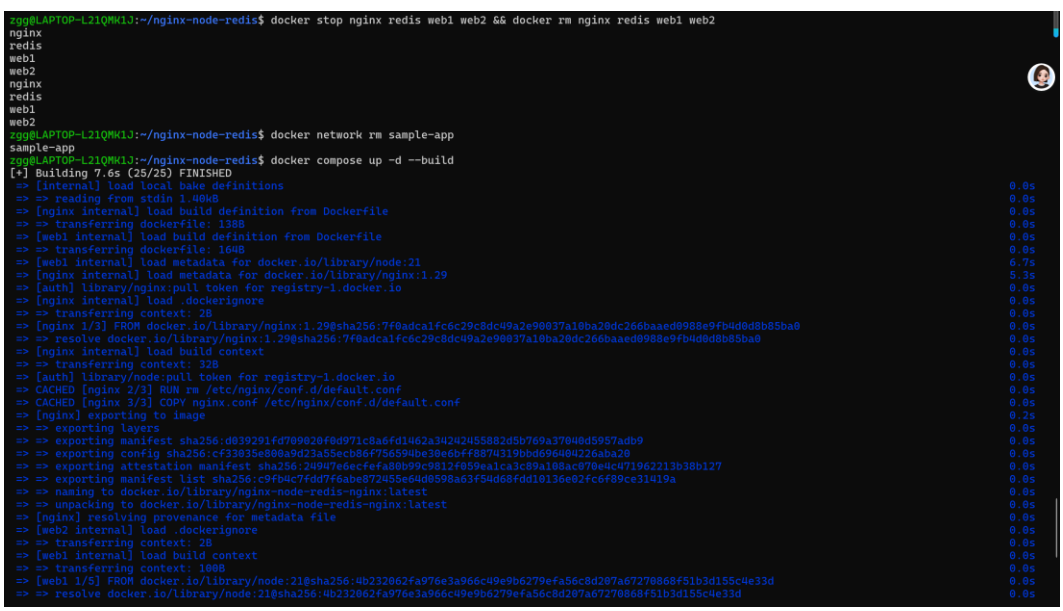

Figure 39: Starting Nginx reverse proxy

5.5 Verifying All Containers

I checked that all containers were running:

docker ps

```
[+] up 8/8
 ✔Image nginx-node-redis-nginx       Built                                                                    7.9s
 ✔Image nginx-node-redis-web1        Built                                                                    7.9s
 ✔Image nginx-node-redis-web2        Built                                                                    7.9s
 ✔Network nginx-node-redis_default   Created                                                                  0.1s
 ✔Container nginx-node-redis-web2-1  Started                                                                  1.2s
 ✔Container nginx-node-redis-redis-1 Started                                                                  1.3s
 ✔Container nginx-node-redis-web1-1  Started                                                                  1.1s
 ✔Container nginx-node-redis-nginx-1 Started                                                                  1.4s
```

Figure 40: All containers running successfully

5.6 Testing the Application

I accessed http://localhost and refreshed several times. The counter incremented, and I could see requests being distributed between web1 and web2:

```
[+] up 8/8
 ✔Image nginx-node-redis-nginx       Built                                                                    7.9s
 ✔Image nginx-node-redis-web1        Built                                                                    7.9s
 ✔Image nginx-node-redis-web2        Built                                                                    7.9s
 ✔Network nginx-node-redis_default   Created                                                                  0.1s
 ✔Container nginx-node-redis-web2-1  Started                                                                  1.2s
 ✔Container nginx-node-redis-redis-1 Started                                                                  1.3s
 ✔Container nginx-node-redis-web1-1  Started                                                                  1.1s
 ✔Container nginx-node-redis-nginx-1 Started                                                                  1.4s
zgg@LAPTOP-L21QMK1J:~/nginx-node-redis$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED             STATUS             PORTS
                                  NAMES
7a57e54befe5   nginx-node-redis-nginx   "/docker-entrypoint.…"   57 seconds ago      Up 55 seconds      0.0.0.0:80->80/tcp, [::]:80->80/tcp
                                  nginx-node-redis-nginx-1
cf4183f5ed77   redis                    "docker-entrypoint.s…"   57 seconds ago      Up 56 seconds      0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
                                  nginx-node-redis-redis-1
e130d7702e3f   nginx-node-redis-web2    "docker-entrypoint.s…"   57 seconds ago      Up 56 seconds      0.0.0.0:82->5000/tcp, [::]:82->5000/tcp
                                  nginx-node-redis-web2-1
29125e8bf3f5   nginx-node-redis-web1    "docker-entrypoint.s…"   57 seconds ago      Up 56 seconds      0.0.0.0:81->5000/tcp, [::]:81->5000/tcp
                                  nginx-node-redis-web1-1
b643546365f7   postgres:18              "docker-entrypoint.s…"   About an hour ago   Up About an hour   0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp
                                  new-db
0cb8ccd9b6c7   gitea/gitea:latest       "/usr/bin/entrypoint…"   5 days ago          Up About an hour   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 0.0.0.0:2
22->22/tcp, [::]:222->22/tcp   gitea
```

Figure 41: Counter application showing load balancing

Nginx distributes requests in a round-robin fashion. The counter value is stored in Redis, so it persists across requests regardless of which web container handles them.

5.7 Using Docker Compose

Managing multiple containers with individual docker run commands is tedious. Docker Compose simplifies this with a single YAML file. I used docker compose up to start everything:

docker compose up -d --build

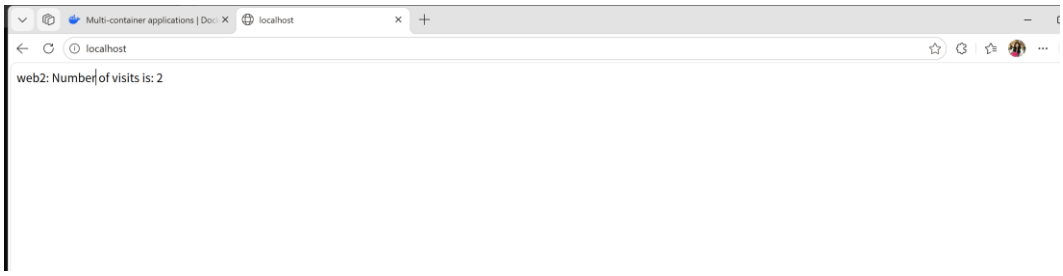

Figure 42: Starting multi-container app with Docker Compose

Docker Compose automatically created the network and started all services in the correct order. This is much more maintainable than running multiple docker run commands.

5.8 Viewing Logs

I could view logs from all services at once:

docker compose logs

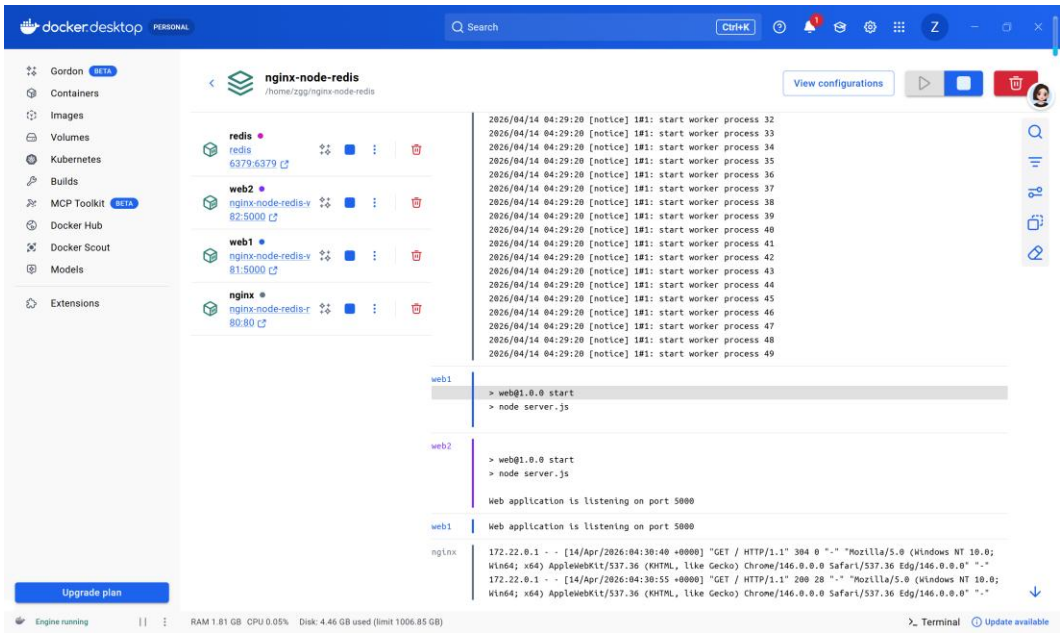

Figure 43: Viewing aggregated logs from all services

5.9 Stopping the Application

Stopping everything is equally simple:

docker compose down

```
zgg@LAPTOP-L21QMK1J:~/nginx-node-redis$ docker compose stop
[+] stop 4/4
 ✔Container nginx-node-redis-redis-1 Stopped                                                    0.8s
 ✔Container nginx-node-redis-nginx-1 Stopped                                                    0.7s
 ✔Container nginx-node-redis-web1-1  Stopped                                                    1.9s
 ✔Container nginx-node-redis-web2-1  Stopped                                                    2.5s
zgg@LAPTOP-L21QMK1J:~/nginx-node-redis$ docker compose down
[+] down 5/5
 ✔Container nginx-node-redis-redis-1 Removed                                                    0.1s
 ✔Container nginx-node-redis-nginx-1 Removed                                                    0.1s
 ✔Container nginx-node-redis-web1-1  Removed                                                    0.2s
 ✔Container nginx-node-redis-web2-1  Removed                                                    0.1s
 ✔Network nginx-node-redis_default   Removed                                                    0.4s
```

Figure 44: Stopping and removing all containers

Conclusion

This lab gave me practical experience with Docker's core features. Here's what I learned:

1. Port Publishing: I can now expose containerized services to the outside world using -p flag. I also learned about ephemeral ports and binding to specific IPs for security.

2. Overriding Defaults: Docker provides many flags to customize container behavior at runtime. Environment variables, network settings, and resource limits are all configurable without modifying images.

3. Data Persistence: Volumes are essential for databases and any application that needs to keep data. I practiced creating volumes, verifying data persistence, and backing up volume data.

4. File Sharing: Bind mounts are perfect for development, allowing real-time file synchronization between host and container. I also learned about permission handling and tmpfs for temporary data.

5. Multi-Container Apps: Docker Compose makes orchestrating complex applications straightforward. I built a working application with Nginx, Node.js, and Redis, all communicating through a custom network.

These skills form the foundation for deploying real-world containerized applications. I now feel confident working with Docker in development and production environments.